

UNIVERSITÀ DEGLI STUDI DI URBINO

DIPARTIMENTO DI SCIENZE PURE E APPLICATE

CORSO DI LAUREA IN INFORMATICA APPLICATA

Progetto di Programmazione Logica e Funzionale

di

Aboufaris Nasrine, Giuseppetti Alessia

Matricola: 321885, 322984

Anno di corso: terzo

Anno Accademico 2024/2025 - Sessione autunnale

Docente: Prof. Marco Bernardo

Indice

1	Specifica del problema	2
2	Analisi del problema	3
2.1	Dati in ingresso del problema	3
2.2	Dati in uscita del problema	3
2.3	Relazioni intercorrenti	3
2.3.1	Relazioni tra dati di ingresso	3
2.3.2	Relazione tra dati di ingresso e dati di uscita	3
3	Progettazione dell'algoritmo	5
3.1	Scelte di progetto	5
3.2	Passi dell'algoritmo	5
4	Implementazione dell'algoritmo	6
4.1	Implementazione in Haskell	6
4.2	Implementazione in Prolog	10
5	Testing	16
5.1	Testing del programma Haskell	16
5.2	Testing del programma Prolog	17

1 Specifica del problema

Scrivere un programma Haskell e un programma Prolog che acquisiscono dalla tastiera un messaggio binario, il quale verrà codificato o decodificato mediante un codice di Hamming generico (n, k) , in base alla scelta dell'utente. I parametri (n, k) saranno derivati da un valore m , fornito dall'utente, che rappresenta il numero di bit di parità. Il programma sarà inoltre in grado di calcolare la distanza di Hamming tra due stringhe binarie, fornite dall'utente, di lunghezza uguale ma arbitraria.

2 Analisi del problema

2.1 Dati in ingresso del problema

I dati in ingresso del problema sono:

- Una scelta tra le operazioni disponibili (codifica, decodifica o calcolo distanza);
- Per la codifica: un numero naturale $m \geq 2$, che rappresenta il numero di bit di parità e una parola binaria di lunghezza $k = 2^m - m - 1$;
- Per la decodifica: un numero naturale $m \geq 2$, che rappresenta il numero di bit di parità e una parola binaria di lunghezza $n = 2^m - 1$;
- Per il calcolo della distanza: due parole binarie di uguale lunghezza $L \geq 1$.

2.2 Dati in uscita del problema

I dati in uscita del problema sono:

- Per la codifica: una parola binaria (composta da sequenze di bit) di lunghezza $n = 2^m - 1$, che rappresenta la parola codificata;
- Per la decodifica: una parola binaria di lunghezza $k = 2^m - m - 1$, che rappresenta la parola decodificata ed eventualmente corretta;
- Per il calcolo della distanza: un numero naturale $d \geq 0$, che rappresenta la distanza di Hamming.

2.3 Relazioni intercorrenti

2.3.1 Relazioni tra dati di ingresso

- Il valore m determina univocamente i parametri del codice: $n = 2^m - 1$ e $k = 2^m - m - 1$;
- La lunghezza delle parole in input deve essere coerente con l'operazione scelta e con il valore m ;
- Per il calcolo della distanza, le due parole devono avere la stessa lunghezza $L \geq 1$.

2.3.2 Relazione tra dati di ingresso e dati di uscita

- I bit di parità vengono inseriti nelle posizioni $p_i = 2^{(i-1)}$ per $i = 1, 2, \dots, m$ (posizioni potenze di due).
- Il valore di ogni bit di parità è calcolato tramite somma modulo 2 (XOR) dei bit che esso controlla. Formalmente:

$$p_i = \bigoplus_{j \in S_i} d_j$$

dove S_i è l'insieme delle posizioni controllate dal bit i -esimo.

- La decodifica utilizza i bit di parità per calcolare il vettore di sindrome S , dove ogni componente è verificata come:

$$s_i = \bigoplus_{j \in C_i} b_j$$

- Se $\text{sindrome} = 0$, non vengono rilevati errori;
 - Se $\text{sindrome} \neq 0$, il valore decimale della sindrome indica la posizione dell'errore singolo.
- La distanza di Hamming tra due parole è pari al numero di posizioni in cui i bit corrispondenti differiscono.

Inoltre è importante indicare determinate proprietà del codice di Hamming:

- Il codice di Hamming (n, k) ha distanza minima pari a 3, garantendo la correzione di errori singoli;
- La correzione è garantita solo per errori singoli, con errori multipli il comportamento non è definito.

3 Progettazione dell'algoritmo

3.1 Scelte di progetto

Le parole binarie verranno rappresentate mediante liste per preservare l'ordine dei bit e sfruttare la natura ricorsiva dei linguaggi scelti. L'approccio adottato privilegia la chiarezza e la correttezza rispetto alla micro-ottimizzazione.

3.2 Passi dell'algoritmo

L'algoritmo segue un approccio ricorsivo:

- **Per la codifica:**

- Acquisire m e calcolare n e k .
- Inserire i bit informativi nelle posizioni non potenze di 2.
- Calcolare ricorsivamente i bit di parità:
 - * *Caso Base*: Se non ci sono più bit di parità da calcolare, restituisci la lista.
 - * *Passo Ricorsivo*: Calcola il bit di parità corrente filtrando le posizioni controllate, aggiorna la lista e procedi con il prossimo bit di parità.

- **Per la decodifica:**

- Calcolare la sindrome verificando le equazioni di parità in modo ricorsivo.
- Se la sindrome è 0, estrarre i dati.
- Se la sindrome $\neq 0$, correggere il bit alla posizione indicata e poi estrarre i dati.

- **Per la distanza di Hamming:**

- *Caso Base*: Se le liste sono vuote, la distanza è 0.
- *Passo Ricorsivo*: Confronta le teste delle liste. Se diverse aggiungi 1, altrimenti 0, e somma al risultato della chiamata ricorsiva sulle code.

4 Implementazione dell'algoritmo

4.1 Implementazione in Haskell

```
1  -- #####
2  -- #      Corso di Programmazione Logica e Funzionale      #
3  -- #      Progetto per la sessione autunnale A.A. 2024/2025      #
4  -- #      di Nasrine Aboufarris, Matricola: 321885      #
5  -- #      e Alessia Giuseppetti, Matricola: 322984      #
6  -- #      Anno di corso: Terzo      #
7  -- #####
8
9  {- Specifica: Scrivere un programma Haskell che acquisisce da tastiera un messaggio binario,
10 il quale verrà codificato o decodificato mediante un codice di Hamming generico (n,k), in base
11 alla scelta dell'utente. I parametri (n,k) saranno derivati da un valore m, fornito dall'
12     ↳ utente,
13 che rappresenta il numero di bit di parità. Il programma sarà inoltre in grado di calcolare la
14 distanza di Hamming tra due stringhe binarie, fornite dall'utente, di lunghezza uguale ma
15     ↳ arbitraria.-}
16
17 import Data.Bits (testBit, xor, (.&.), shiftL)
18 import Text.Read (readMaybe)
19 import Control.Monad (unless, when)
20 import Data.List (foldl')
21 import System.Exit (exitSuccess)
22
23 -- Tipi di dati per migliorare la leggibilità del codice
24 type Bit = Int
25 type ParolaBinaria = [Bit]
26
27 {- main: Funzione principale del programma -}
28 main :: IO ()
29 main = do
30     print "-----Benvenuto-----"
31     print "Il seguente programma permette di utilizzare i codici di Hamming generici (n,k)"
32     print "per la codifica, decodifica e calcolo della distanza di Hamming tra due parole"
33     print "binarie."
34     print "Come funziona il programma:"
35     print "Potrai scegliere dal menu una delle tre operazioni da eseguire, per la codifica"
36     print "e decodifica di Hamming, verrà chiesto di inserire m, ovvero il numero di bit "
37     print "di parità, grazie ad m verranno calcolati n e k, e si potrà inserire la"
38     print "parola binaria. Esempio di parola accetta '1011'"
39     print "Notare che per la decodifica questo algoritmo corregge solo errori singoli."
40     print "Con errori multipli il risultato potrebbe essere errato."
41     print "Il programma finisce solo nel momento in cui si sceglie l'opzione esci dal menu"
42     print "-----"
43     cicloPrincipale
44
45 {- cicloPrincipale: Gestisce il menu principale e le scelte dell'utente -}
46 cicloPrincipale :: IO ()
47 cicloPrincipale = do
48     -- Lista delle operazioni disponibili nel programma
49     let vociMenu = [ "Codifica di Hamming"
50                     , "Decodifica di Hamming"
51                     , "Distanza di Hamming"
52                     , "Esci"
53                     ]
54
55     -- Stampa il menu usando mapM_ per eseguire l'azione su ogni elemento
56     putStrLn "\nScegli operazione:"
57     mapM_ (\(indice, voce) -> putStrLn $ show indice ++ " - " ++ voce)
58         (zip [1..] vociMenu)
59
60     -- Pattern matching sull'input dell'utente per determinare l'azione
61     scelta <- getLine
62     case scelta of
63         "1" -> gestioneCodificaHamming >> cicloPrincipale
64         "2" -> gestioneDecodificaHamming >> cicloPrincipale
65         "3" -> gestioneDistanzaHamming >> cicloPrincipale
66         "4" -> exitSuccess
67         _    -> do
68             putStrLn "Scelta non valida. Riprova."
69             cicloPrincipale
70
71 {- gestioneCodificaHamming: Gestisce il processo di codifica di Hamming
```

```

70  coordina input utente, calcoli e output per la codifica -}
71  gestioneCodificaHamming :: IO()
72  gestioneCodificaHamming = do
73      putStrLn "\nCodifica di Hamming"
74      parametroM <- controlloParametroM
75      -- Calcola automaticamente n e k basandosi su m
76      let (parametroN, parametroK) = calcolaParametriHamming parametroM
77      putStrLn $ "Parametri calcolati: n = " ++ show parametroN ++ ", k = " ++ show parametroK
78      parola <- controlloParolaBinaria parametroK
79      {- Converte la stringa in lista di bit usando map e read -}
80      let parolaDati = map (read . (:[])) parola :: ParolaBinaria
81      let parolaCodificata = codificaHamming parametroM parolaDati
82      putStrLn $ "La parola codificata è: " ++ concatMap show parolaCodificata
83
84  {- gestioneDecodificaHamming: Gestisce il processo di decodifica di Hamming
85     coordina input utente, calcoli e output per la decodifica -}
86  gestioneDecodificaHamming :: IO()
87  gestioneDecodificaHamming = do
88      putStrLn "\nDecodifica di Hamming"
89      parametroM <- controlloParametroM
90      let (parametroN, parametroK) = calcolaParametriHamming parametroM
91      putStrLn $ "Parametri calcolati: n = " ++ show parametroN ++ ", k = " ++ show parametroK
92      parola <- controlloParolaBinaria parametroN
93      let parolaRicevuta = map (read . (:[])) parola :: ParolaBinaria
94      let (parolaDecodificata, errore) = decodificaHamming parametroM parolaRicevuta
95      -- Pattern matching su Maybe per gestire presenza o assenza di errore
96      case errore of
97          Nothing ->
98              putStrLn $ "Nessun errore rilevato. La parola decodificata è: " ++ concatMap show
99                  ↪ parolaDecodificata
100             Just posizione ->
101                 putStrLn $ "Corretto un errore nella posizione " ++ show posizione ++
102                     ". La parola decodificata è: " ++ concatMap show parolaDecodificata
103
104  {- gestioneDistanzaHamming: Gestisce il calcolo della distanza di Hamming
105     coordina input di due parole e calcolo della distanza -}
106  gestioneDistanzaHamming :: IO()
107  gestioneDistanzaHamming = do
108      putStrLn "\nDistanza di Hamming"
109      (s1, s2) <- controlloDueParole
110      let p1 = map (read . (:[])) s1 :: ParolaBinaria
111      let p2 = map (read . (:[])) s2 :: ParolaBinaria
112      putStrLn $ "La distanza di Hamming tra le due parole è: " ++ show (calcolaDistanzaHamming
113          ↪ p1 p2)
114
115  {- controlloParametroM: Valida l'input del parametro m
116     continua a chiedere input fino a ottenere un valore valido -}
117  controlloParametroM :: IO Int
118  controlloParametroM = do
119      putStrLn "Inserisci m (m >= 2):"
120      input <- getLine
121      -- Usa readMaybe per parsing sicuro che ritorna Maybe Int
122      case readMaybe input of
123          Just m | m >= 2 -> return m
124          _ -> do
125              putStrLn "Errore: m deve essere un intero maggiore uguale a 2"
126              controlloParametroM
127
128  {- controlloParolaBinaria: Valida l'input di una parola binaria di lunghezza specifica
129     Argomenti: lunghezza richiesta della parola
130     continua a chiedere input fino a ottenere una parola valida -}
131  controlloParolaBinaria :: Int -> IO String
132  controlloParolaBinaria lunghezza = do
133      putStrLn $ "Inserisci parola binaria di lunghezza " ++ show lunghezza ++ ":"
134      parola <- getLine
135      -- all verifica che tutti i caratteri siano in "01"
136      if all ('elem' "01") parola && length parola == lunghezza
137          then return parola
138          else do
139              putStrLn "Errore: lunghezza non valida o caratteri non binari"
140              controlloParolaBinaria lunghezza
141
142  {- controlloDueParole: Valida l'input di due parole binarie di uguale lunghezza
143     continua a chiedere input fino a ottenere due parole valide -}
144  controlloDueParole :: IO (String, String)
145  controlloDueParole = do

```

```

144 putStrLn "Inserisci prima parola binaria:"
145 p1 <- getLine
146 putStrLn "Inserisci seconda parola binaria (stessa lunghezza):"
147 p2 <- getLine
148 if length p1 == length p2 && all ('elem' "01") p1 && all ('elem' "01") p2
149   then return (p1, p2)
150   else do
151     putStrLn "Errore: le parole devono essere binarie e di uguale lunghezza"
152     controlloDueParole
153
154 {- calcolaParametriHamming: Calcola i parametri n e k del codice di Hamming
155    Argomenti: m (numero di bit di parità)
156    calcola n = 2^m - 1 e k = n - m -}
157 calcolaParametriHamming :: Int -> (Int, Int)
158 calcolaParametriHamming m = (n, n - m)
159   -- where permette di definire n localmente per riutilizzarlo
160   where n = 2 ^ m - 1
161
162 {- potenzaDiDue: Verifica se un numero è una potenza di 2
163    Argomenti: n (numero da verificare) -}
164 potenzaDiDue :: Int -> Bool
165 potenzaDiDue n = n /= 0 && (n .&. (n-1)) == 0
166
167 {- aggiornaElemento: Aggiorna un elemento in una lista a un indice specifico
168    Argomenti: lista, indice, nuovo valore
169    divide la lista e ricostruisce con nuovo valore -}
170 aggiornaElemento :: [a] -> Int -> a -> [a]
171 aggiornaElemento lista indice nuovo =
172   -- splitAt divide la lista in due parti all'indice specificato
173   let (prima, _:dopo) = splitAt indice lista
174   in prima ++ [nuovo] ++ dopo
175
176 {- codificaHamming: Codifica una parola usando il codice di Hamming
177    Argomenti: m (bit di parità), parolaDati (dati da codificare)
178    crea parola iniziale e inserisce dati nelle posizioni corrette
179    coordina inserimento dati e calcolo bit di parità -}
180 codificaHamming :: Int -> ParolaBinaria -> ParolaBinaria
181 codificaHamming m parolaDati =
182   let (n, _) = calcolaParametriHamming m
183   -- Inizializza una parola di tutti zeri
184   parolaIniziale = replicate n 0
185   indiciNonParita = filter (\j -> not (potenzaDiDue (j+1))) [0..n-1]
186   parolaConDati = inserisciDati parolaIniziale indiciNonParita parolaDati
187   in calcolaParita parolaConDati n m
188
189 {- calcolaParita: Calcola tutti i bit di parità per una parola
190    Argomenti: parola, n (lunghezza), m (numero bit parità)
191    Casi base: quando lista [0..m-1] è vuota ritorna parola invariata
192    Casi generali: per ogni i in [0..m-1] calcola il bit di parità i-esimo
193    usa foldl' per applicare calcolaEAggiornaParita a ogni posizione di parità -}
194 calcolaParita :: ParolaBinaria -> Int -> Int -> ParolaBinaria
195 calcolaParita parola n m = foldl' (calcolaEAggiornaParita n) parola [0..m-1]
196
197
198 {- calcolaEAggiornaParita: Calcola e aggiorna un singolo bit di parità
199    Argomenti: n (lunghezza parola), parola, i (indice bit parità)
200    identifica bit controllati, calcola XOR e aggiorna posizione -}
201 calcolaEAggiornaParita :: Int -> ParolaBinaria -> Int -> ParolaBinaria
202 calcolaEAggiornaParita n parola i =
203   let posizioneParita = 2^i
204   indiceParita = posizioneParita - 1
205   -- testBit j i verifica se il bit i-esimo di j è settato
206   indiciControllati = [ j-1 | j <- [1..n], testBit j i ]
207   -- Esclude il bit di parità stesso dal calcolo -}
208   bitDati = map (parola !!) (filter (/= indiceParita) indiciControllati)
209   -- XOR di tutti i bit controllati
210   valoreXOR = foldl' xor 0 bitDati
211   in aggiornaElemento parola indiceParita valoreXOR
212
213 {- inserisciDati: Inserisce i dati nelle posizioni specificate
214    Argomenti: parola iniziale, lista indici, lista dati
215    Casi base: quando liste indici e dati sono vuote ritorna parola invariata
216    Casi generali: per ogni coppia (indice, dato) aggiorna la parola
217    usa foldl' per applicare aggiornaElemento a ogni coppia -}
218 inserisciDati :: [a] -> [Int] -> [a] -> [a]
219 inserisciDati parola indici dati =

```

```

220 -- zip associa ogni indice al dato corrispondente
221 foldl' (\acc (indice, dato) -> aggiornaElemento acc indice dato) parola (zip indici dati)
222
223 {- calcolaSindrome: Calcola la sindrome per rilevare errori
224 Argomenti: m (bit parità), parola ricevuta
225 Casi base: quando lista [0..m-1] è vuota sindrome = 0
226 Casi generali: per ogni bit di parità verifica correttezza e accumula errori
227 usa foldl' per verificare ogni bit di parità e costruire sindrome -}
228 calcolaSindrome :: Int -> ParolaBinaria -> Int
229 calcolaSindrome m parola =
230     let n = length parola
231     in foldl' (\sindrome i ->
232         let posizioneParita = 2i
233             indiceParita = posizioneParita - 1
234             indiciControllati = [ j-1 | j <- [1..n], testBit j i ]
235             bitDati = map (parola !!) (filter (/= indiceParita) indiciControllati)
236             valoreAtteso = foldl' xor 0 bitDati
237             valoreParita = parola !! indiceParita
238             {- Se la parità non corrisponde, aggiungi la posizione alla sindrome -}
239             in if valoreParita /= valoreAtteso
240                 then sindrome + posizioneParita
241                 else sindrome
242         ) 0 [0..m-1]
243
244 {- estraiDati: Estrae i bit di dati da una parola codificata
245 Argomenti: m (bit parità), parola codificata
246 filtra posizioni non-parità e mappa gli elementi -}
247 estraiDati :: Int -> ParolaBinaria -> ParolaBinaria
248 estraiDati m parola =
249     let n = length parola
250     in let indiciDati = filter (\i -> not (potenzaDiDue (i+1))) [0..n-1]
251        in map (parola !!) indiciDati
252
253 {- decodificaHamming: Decodifica una parola e corregge un eventuale errore
254 Argomenti: m (bit parità), parola ricevuta
255 calcola sindrome e decide se correggere o meno -}
256 decodificaHamming :: Int -> ParolaBinaria -> (ParolaBinaria, Maybe Int)
257 decodificaHamming m parola =
258     let sindrome = calcolaSindrome m parola
259     in if sindrome == 0
260         then (estraiDati m parola, Nothing)
261         else
262             let posizioneErrore = sindrome - 1
263             parolaCorretta = aggiornaElemento parola posizioneErrore (1 - parola !!
264                 ↪ posizioneErrore)
265             in (estraiDati m parolaCorretta, Just (posizioneErrore + 1))
266
267 {- calcolaDistanzaHamming: Calcola la distanza di Hamming tra due parole
268 Argomenti: parola1, parola2 -}
269 calcolaDistanzaHamming :: ParolaBinaria -> ParolaBinaria -> Int
270 calcolaDistanzaHamming parola1 parola2 =
271     sum (zipWith (\a b -> if a == b then 0 else 1) parola1 parola2)

```

Listing 1: Codice Sorgente Haskell

4.2 Implementazione in Prolog

```
1  /* #####
2  # Corso di Programmazione Logica e Funzionale #
3  # Progetto per la sessione autunnale A.A. 2024/2025 #
4  # di Nasrine Aboufaris, Matricola: 321885 #
5  # e Alessia Giuseppetti, Matricola: 322984 #
6  # Anno di corso: Terzo #
7  #####
8  Specifica: Programma Prolog per la codifica/decodifica #
9  di codici di Hamming e calcolo della distanza di Hamming */
10
11 /* Predicato principale del programma. Fornisce istruzioni all'utente per l'uso del programma.
12    ↪ */
13 main :- write('-----Benvenuto-----'), nl,
14         write('Il seguente programma permette di utilizzare i codici di Hamming generici (n,k)
15         ↪ per '), nl,
16         write('la codifica, decodifica e calcolo della distanza di Hamming tra due parole
17         ↪ binarie. '), nl,
18         write('Si ricorda che la decodifica di Hamming di seguito implementata permette solo
19         ↪ di '), nl,
20         write('Individuare e correggere un solo errore. '), nl,
21         write('Come funziona il programma: '), nl,
22         write(' 1. Ti verra\' richiesto il numero di bit di parita\' m. '), nl,
23         write(' 2. Da m verranno calcolati automaticamente n e k. '), nl,
24         write(' 3. Segui le istruzioni del menu per completare l\'operazione desiderata. '),
25         ↪ nl,
26         write('La parola deve essere inserita come una lista di interi, '), nl,
27         write('si prega di usare la seguente formattazione: '), nl,
28         write('Scrivere la parola tra parentesi quadre e inserire una virgola tra un numero e
29         ↪ l\'altro. '), nl,
30         write('Esempio di parola accettata: [1, 0, 1, 0]. '), nl,
31         write('Esempio di parola non accettata 1010. '), nl,
32         write('Si prega di inserire il punto finale prima dell\'invio e si raccomanda di
33         ↪ seguire questa formattazione '), nl,
34         write('per evitare anomalie durante l\'esecuzione del programma. '), nl,
35         write('-----'), nl,
36         menu.
37
38 /* Il predicato menu mostra le opzioni disponibili e
39    acquisisce tramite tastiera la scelta dell'utente. */
40 menu :- write('Scegliere l\'operazione che si desidera eseguire: '), nl,
41         write(' 1. Codifica di Hamming '), nl,
42         write(' 2. Decodifica di Hamming '), nl,
43         write(' 3. Distanza di Hamming '), nl,
44         write(' 4. Esci '), nl,
45         read(Opzione),
46         esegui_opzione(Opzione).
47
48 /* Il predicato esegui_opzione(1) esegue la codifica di Hamming. Chiede il numero di bit di
49    ↪ parità M,
50    calcola il parametro k, valida la parola input P di lunghezza k ed esegue la codifica.
51    - Il primo argomento è l'opzione 1 della scelta dell'utente. */
52 esegui_opzione(1) :- parita(M),
53                     parametri_hamming(M, K, _N),
54                     acquisisci_parola(Parola, K),
55                     write('Esecuzione codifica... '), nl,
56                     codifica_parola(Parola, M),
57                     menu.
58
59 /* Il predicato esegui_opzione(2) esegue la Decodifica di Hamming. Chiede il numero di bit di
60    ↪ parità M,
61    calcola il parametro n, valida il codice ricevuto di lunghezza n ed esegue la decodifica.
62    - Il primo argomento è l'opzione 2 della scelta dell'utente. */
63 esegui_opzione(2) :- parita(M),
64                     parametri_hamming(M, _K, N),
65                     acquisisci_parola(Parola, N),
66                     write('Esecuzione decodifica... '), nl,
67                     decodifica_parola(Parola, M),
68                     menu.
69
70 /* Il predicato esegui_opzione(3) calcola la distanza di Hamming tra due parole.
71    Acquisisce due parole binarie, ne verifica la validità, controlla che abbiano la stessa
72    ↪ lunghezza
73    e calcola la distanza.
74    - Il primo argomento è l'opzione 3 scelta dall'utente. */
```

```

65 esegui_opzione(3) :- write('Inserire la prima parola: '), nl,
66     acquisisci_parola_distanza(T1),
67     write('Inserire la seconda parola: '), nl,
68     acquisisci_parola_distanza(T2),
69     confronto_lunghezza(T1, T2),
70     calcolo_distanza(T1, T2),
71     menu.
72
73 /* Il predicato esegui_opzione(4) si occupa di terminare il programma.
74    Termina l'esecuzione del programma dopo aver stampato un messaggio di saluto.
75    - Il primo argomento è l'opzione 4 scelta dall'utente. */
76 esegui_opzione(4) :- write('Grazie per aver utilizzato il programma. Arrivederci!'), nl,
77     halt. % Termina il programma
78
79 /* Il predicato esegui_opzione(_) gestisce opzioni non valide.
80    Visualizza un messaggio di errore e torna al menu principale.
81    - Il primo argomento è un'opzione non compresa tra 1 e 4. */
82 esegui_opzione(_) :- write('Opzione non valida. Riprovare.'), nl,
83     menu.
84
85 /* Il predicato parita(M) acquisisce da tastiera il numero di bit di parità scelto
86    dall'utente e ne verifica la validità, assicurandosi che il numero
87    sia maggiore o uguale a 2. */
88 parita(M) :- nl, write('Il bit di parita\' deve essere maggiore o uguale a 2. '), nl,
89     write('Inserire il bit di parita\' scelto:'), nl,
90     % lettura dell'input
91     (catch(read(M), _, fail),
92     % validazione dell'input
93     integer(M),
94     M >= 2
95     ->
96     true
97     ;
98     nl, write('Input non valido! Deve essere un intero >= 2.'), nl,
99     parita(M)
100    ).
101
102 /* Il predicato parametri_hamming(M, K, N) calcola i parametri k(bit di dati) ed n(bit totali)
103    per il codice di Hamming.
104    - Il primo argomento rappresenta il numero di bit di parità.
105    - Il secondo argomento rappresenta il numero di bit di dati.
106    - Il terzo argomento rappresenta il numero totale di bit.
107    Esempio: parametri_hamming(3, K, N) calcola K=4 e N=7. */
108 parametri_hamming(M,K,N):- K is 2^M - 1 - M,
109     N is 2^M -1.
110
111 /* Il predicato acquisisci_parola acquisisce una parola binaria dall'input
112    e la valida rispetto alla lunghezza specificata.
113    - Il primo argomento è la parola validata in output.
114    - Il secondo argomento è la lunghezza attesa della parola. */
115 acquisisci_parola(Parola, Lunghezza) :- write('Inserire la parola di lunghezza '),
116     write(Lunghezza),
117     write(': '),
118     (catch(read(Input), _, fail)
119     -> elabora_input(Input, Lunghezza, Parola)
120     ; nl, write('Errore di acquisizione!'), nl,
121     acquisisci_parola(Parola, Lunghezza)
122    ).
123
124 /* Il predicato elabora_input gestisce la validazione dell'input e la ricorsione
125    in caso di errore. Se l'input è valido, unifica Parola con Input; altrimenti,
126    stampa un messaggio di errore e richiede nuovo input.
127    - Il primo argomento è l'input letto da tastiera.
128    - Il secondo argomento è la lunghezza attesa.
129    - Il terzo argomento è l'output validato. */
130 elabora_input(Input, Lunghezza, Parola) :- (valida_parola_binaria(Input, Lunghezza)
131     -> Parola = Input
132     ; write('Parola non valida. Deve essere lunga ')
133
134     write(Lunghezza),
135     write(' e contenere solo 0 e 1.'), nl,
136     acquisisci_parola(Parola, Lunghezza)
137    ).
138
139

```

```

140 /* Il predicato valida_parola_binaria verifica che una lista sia valida per il codice di
    ↪ Hamming, controllando che sia
141 una lista, che abbia che abbia lunghezza Lunghezza e che tutti gli elementi siano bit
    ↪ validi.
142 - il primo argomento è la lista da verificare
143 - il secondo argomento è la lunghezza della parola attesa. */
144 valida_parola_binaria(Parola, Lunghezza) :- is_list(Parola),
145                                             length(Parola, Lunghezza),
146                                             maplist(bit_valido, Parola).
147
148 /* Il fatto bit_valido definisce i bit validi per il codice di Hamming.
149 Un bit è valido se è 0 o 1.
150 - L'unico argomento rappresenta il bit da verificare. */
151 bit_valido(0).
152 bit_valido(1).
153
154 /* Il predicato acquisisci_parola_distanza acquisisce una parola binaria generica dall'input
155 e la valida.
156 - L'unico argomento è la parola validata in output. */
157 acquisisci_parola_distanza(Parola) :- (catch(read(Input), _, fail),
158                                     elabora_input_distanza(Input, Parola)
159                                     ; nl, write('Errore di acquisizione!'), nl,
160                                     acquisisci_parola_distanza(Parola)
161                                     ).
162
163 /* Il predicato elabora_input_distanza gestisce la validazione dell'input e la ricorsione
164 in caso di errore. Se l'input è valido, unifica Parola con Input; altrimenti,
165 stampa un messaggio di errore e richiede nuovo input.
166 - Il primo argomento Input è l'input letto da tastiera.
167 - Il secondo argomento Parola è l'output validado. */
168 elabora_input_distanza(Input, Parola) :- (validazione_parola_distanza(Input),
169                                     Parola = Input
170                                     ; write('Parola non valida. Deve essere una lista
    ↪ contenente solo 0 e 1.'), nl,
171                                     acquisisci_parola_distanza(Parola)
172                                     ).
173
174 /* Il predicato validazione_parola_generica verifica che l'input sia una lista
175 composta esclusivamente da valori binari (0 e 1).
176 - L'unico argomento rappresenta la parola da validare. */
177 validazione_parola_distanza(Parola) :- is_list(Parola),
178                                     maplist(bit_valido, Parola).
179
180 /* Il predicato confronto_lunghezza verifica che due liste abbiano la stessa lunghezza,
181 condizione necessaria per il calcolo della distanza di Hamming.
182 - Il primo argomento rappresenta la prima lista da confrontare.
183 - Il secondo argomento rappresenta la seconda lista da confrontare. */
184 confronto_lunghezza(T1, T2) :- length(T1, L1),
185                                length(T2, L2),
186                                (L1 == L2
187                                -> true
188                                ; write('Le due parole devono avere la stessa lunghezza.'), nl
    ↪ ,
189                                write('Prima parola lunghezza: '), write(L1), nl,
190                                write('Seconda parola lunghezza: '), write(L2), nl,
191                                write('Inserire nuovamente le due parole.'), nl,
192                                esegui_opzione(3)
193                                ).
194
195 /* Il predicato posizione_parita calcola, dato un indice, se la posizione è potenza di due o
    ↪ meno.
196 - L'unico argomento indica la posizione che gli viene passata. */
197 posizione_parita(Posizione) :- Posizione > 0,
198                                (Posizione /\ (Posizione - 1)) == 0.
199
200 /* Il predicato inserisci_bit inserisce bit di dati nelle posizioni non di parità di un codice
    ↪ Hamming,
201 lasciando dei segnaposti per i bit di parità.
202 - Il primo argomento rappresenta la posizione corrente nel codice.
203 - Il secondo argomento è la lista di bit di dati da inserire.
204 - Il terzo argomento è il codice risultante con i bit di dati nelle posizioni appropriate.
    ↪ */
205 inserisci_bit(_, [], []) :- !.
206 inserisci_bit(Pos, [B|Coda], [B|Resto]) :- \+ posizione_parita(Pos),
207                                         PosSucc is Pos + 1,
208                                         inserisci_bit(PosSucc, Coda, Resto).

```

```

209 inserisci_bit(Pos, BitDati, [_|Resto]) :- posizione_parita(Pos),
210                                         PosSucc is Pos + 1,
211                                         inserisci_bit(PosSucc, BitDati, Resto).
212
213 /* Il predicato calcolo_parita calcola il valore dei bit di parità per una specifica posizione
   ↳ P
214    in una lista di bit, utilizzando l'operazione XOR tra i bit nelle posizioni appropriate.
215    - Il primo argomento è la lista di bit su cui operare.
216    - Il secondo argomento indica la posizione del bit di parità da calcolare.
217    - Il terzo argomento è il valore calcolato del bit di parità. */
218 calcolo_parita(Lista, Pos, Valore_P) :- calcolo_parita(Lista, Pos, 1, 0, Valore_P).
219
220 /* Il predicato calcolo_parita implementa il calcolo ricorsivo del bit di parità.
221    - Il primo argomento è la lista di bit rimanente da elaborare.
222    - Il secondo argomento è la posizione del bit di parità.
223    - Il terzo argomento è la posizione corrente nella lista.
224    - Il quarto argomento è l'accumulatore per il calcolo XOR.
225    - Il quinto argomento è il valore finale del bit di parità. */
226 calcolo_parita([], _, _, Acc, Acc).
227 calcolo_parita([Bit|Resto], Pos, Indice, Acc, Valore_P) :- ((Indice /\ Pos) > 0 ->
228                                                         xor_bit(Acc, Bit, NuovoAcc)
229                                                         ;   NuovoAcc = Acc
230                                                         ),
231                                                         ProssimoIndice is Indice + 1,
232                                                         calcolo_parita(Resto, Pos,
233                                                         ↳ ProssimoIndice, NuovoAcc, Valore_P).
234
235 /* Fatto xor_bit implementa l'operazione XOR tra due bit.
236    - Il primo argomento è il primo bit.
237    - Il secondo argomento è il secondo bit.
238    - Il terzo argomento è il risultato dell'operazione XOR. */
239 xor_bit(0, 0, 0).
240 xor_bit(0, 1, 1).
241 xor_bit(1, 0, 1).
242 xor_bit(1, 1, 0).
243
244 /* Il predicato sostituisci_elemento sostituisce l'elemento in una specifica posizione di una
   ↳ lista.
245    - Il primo argomento è la lista originale.
246    - Il secondo argomento è la posizione dell'elemento da sostituire.
247    - Il terzo argomento è il nuovo valore da inserire.
248    - Il quarto argomento è la lista risultante dopo la sostituzione. */
249 sostituisci_elemento([_|T], 1, X, [X|T]).
250 sostituisci_elemento([H|T], Pos, X, [H|T1]) :- Pos > 1,
251                                                  Pos1 is Pos - 1,
252                                                  sostituisci_elemento(T, Pos1, X, T1).
253
254 /* Il predicato calcola_tutte_parita calcola tutti i bit di parità per una lista di posizioni
   e li inserisce nel codice.
255    - Il primo argomento è il codice temporaneo senza bit di parità.
256    - Il secondo argomento è la lista delle posizioni dei bit di parità.
257    - Il terzo argomento è il codice completo con tutti i bit di parità calcolati. */
258 calcola_tutte_parita(List, [], List).
259 calcola_tutte_parita(List, [P|Resto], Risultato) :- calcolo_parita(List, P, PV),
260                                                         sostituisci_elemento(List, P, PV,
261                                                         ↳ NuovaLista),
262                                                         calcola_tutte_parita(NuovaLista, Resto,
263                                                         ↳ Risultato).
264
265 /* Il predicato parita_list genera la lista delle posizioni dei bit di parità per un dato
   ↳ numero M.
266    Le posizioni sono le potenze di 2 fino a 2^(M-1).
267    - Il primo argomento è il numero di bit di parità.
268    - Il secondo argomento è la lista delle posizioni dei bit di parità. */
269 parita_list(M, List) :- M1 is M - 1,
270                         findall(P, (between(0, M1, Exp), P is 2^Exp), List).
271
272 /* Il predicato codifica_parola implementa la codifica di Hamming per una parola di dati.
273    - Il primo argomento è la parola di dati da codificare.
274    - Il secondo argomento è il numero di bit di parità.
275    Il predicato inserisce i bit di dati nelle posizioni appropriate,
276    calcola i bit di parità e produce il codice di Hamming finale. */
277 codifica_parola(Parola, M) :- % Calcola le posizioni dei bit di parità
278                               parita_list(M, PosizioniParita),

```

```

279                                     % Inserisce i bit di dati nelle posizioni appropriate
280                                     inserisci_bit(1, Parola, CodiceTemp),
281                                     % Calcola i valori dei bit di parità e li inserisce
282                                     calcola_tutte_parita(CodiceTemp, PosizioniParita, CodiceHamming
    ↪ ),
283                                     write('Codice Hamming: '),
284                                     write(CodiceHamming), nl.
285
286
287
288 /* Il predicato calcola_sindrome calcola la sindrome per un codice Hamming ricevuto.
289 La sindrome rappresenta la somma delle posizioni dei bit di parità che risultano errati.
290 Se la sindrome è 0, non ci sono errori, in caso contrario indica la posizione dell'errore.
291 - Il primo argomento è il codice di Hamming da analizzare.
292 - Il secondo argomento è la lista delle posizioni dei bit di parità.
293 - Il terzo argomento è il valore calcolato della sindrome. */
294 calcola_sindrome(_, [], 0).
295 calcola_sindrome(Parola, [P|Resto], Sindrome) :- calcolo_parita(Parola, P, Valore_P),
296                                     calcola_sindrome(Parola, Resto,
    ↪ SindromeParziale),
297                                     (Valore_P == 1 ->
298                                     Sindrome is SindromeParziale + P
299                                     ; Sindrome = SindromeParziale).
300
301
302 /* Il predicato elemento_in restituisce l'elemento in una specifica posizione di una lista
303 utilizzando la numerazione basata su 1 (il primo elemento è in posizione 1).
304 - Il primo argomento è la lista in cui cercare.
305 - Il secondo argomento è la posizione dell'elemento da estrarre.
306 - Il terzo argomento è l'elemento trovato nella posizione specificata. */
307 elemento_in([X|_], 1, X).
308 elemento_in([_|T], Pos, X) :- Pos > 1,
309                                     Pos1 is Pos - 1,
310                                     elemento_in(T, Pos1, X).
311
312 /* Il predicato correggi_errore corregge un singolo errore bit in un codice di Hamming
313 invertendo il valore del bit nella posizione specificata (0 diventa 1, 1 diventa 0).
314 - Il primo argomento è il codice originale contenente l'errore.
315 - Il secondo argomento è la posizione del bit errato.
316 - Il terzo argomento è il codice con l'errore corretto. */
317 correggi_errore(Lista, Pos, ListaCorretta) :- elemento_in(Lista, Pos, Bit),
318                                     (Bit == 0 -> NuovoBit = 1 ; NuovoBit = 0),
319                                     sostituisci_elemento(Lista, Pos, NuovoBit,
    ↪ ListaCorretta).
320
321 /* Il predicato estrai_dati estrae i bit di dati da un codice di Hamming,
322 rimuovendo i bit di parità che si trovano nelle posizioni che sono potenze di 2.
323 - Il primo argomento è il codice Hamming completo.
324 - Il secondo argomento è la lista contenente solo i bit di dati. */
325 estrai_dati(Codice, Dati) :- estrazione_dati(Codice, 1, Dati).
326
327 /* Predicato ricorsivo per estrai_dati che effettua l'estrazione effettiva
328 dei bit di dati, saltando i bit di parità nelle posizioni che sono potenze di 2.
329 - Il primo argomento è la porzione rimanente del codice da elaborare.
330 - Il secondo argomento è la posizione corrente nel codice (contatore).
331 - Il terzo argomento è la lista accumulata dei bit di dati estratti. */
332 estrazione_dati([], _, []).
333 estrazione_dati([Bit|Resto], Pos, [Bit|DatiResto]) :- \+ posizione_parita(Pos),
334                                     !,
335                                     PosSucc is Pos + 1,
336                                     estrazione_dati(Resto, PosSucc,
    ↪ DatiResto).
337 estrazione_dati([_|Resto], Pos, Dati) :- posizione_parita(Pos),
338                                     !,
339                                     PosSucc is Pos + 1,
340                                     estrazione_dati(Resto, PosSucc, Dati).
341
342 /* Il predicato decodifica_parola implementa la decodifica completa di un codice di Hamming.
343 Calcola la sindrome del codice ricevuto: se è zero non ci sono errori, altrimenti
344 corregge l'errore nella posizione indicata e estrae i bit di dati.
345 - Il primo argomento CodiceRicevuto è il codice Hamming da decodificare.
346 - Il secondo argomento M è il numero di bit di parità utilizzati. */
347 decodifica_parola(CodiceRicevuto, M) :- parita_list(M, PosizioniParita),
348                                     calcola_sindrome(CodiceRicevuto, PosizioniParita,
    ↪ Sindrome),
349                                     (Sindrome == 0 ->

```

```

350         write('Nessun errore trovato. '), nl,
351         estrai_dati(CodiceRicevuto, Dati)
352     ; write('Errore trovato in posizione: '), write(
    ↳ Sindrome), nl,
353     ↳ CodiceCorretto),
354         estrai_dati(CodiceCorretto, Dati)
355     ),
356     write('Parola decodificata: '), write(Dati), nl.
357
358 /* Predicato che esegue il calcolo di Hamming in maniera ricorsiva.
359    - il primo argomento indica la prima lista inserita;
360    - il secondo argomento indica la seconda lista inserita;
361    - il terzo argomento è la distanza di Hamming risultante.
362    Il calcolo della distanza è stato fatto in maniera ricorsiva:*/
363
364 calcolo_distanza([], [], 0).
365 calcolo_distanza([H1|T1], [H2|T2], D) :- calcolo_distanza(T1, T2, D1),
366     (   H1 == H2
367     ->  D = D1
368     ;   D is D1 + 1
369     ).
370 calcolo_distanza(T1, T2) :- calcolo_distanza(T1, T2, D),
371     write('Distanza di Hamming: '), write(D), nl.

```

Listing 2: Codice Sorgente Prolog

5 Testing

5.1 Testing del programma Haskell

Test nr. 1 - Errore nella scelta del menu

```
"-----Benvenuto-----"
...
Scegli operazione:
1 - Codifica di Hamming
...
scelta: 5
Scelta non valida. Riprova.
```

Test nr. 2 - Errore m minore di 2

```
...
Inserisci m (m >= 2): 1
Errore: m deve essere un intero maggiore uguale a 2
```

Test nr. 3 - Errore lunghezza parola codifica

```
...
Inserisci parola binaria di lunghezza 4: 10000
Errore: lunghezza non valida o caratteri non binari
```

Test nr. 4 - Codifica con $m = 3$

```
...
Inserisci m (m >= 2): 3
Parametri calcolati: n = 7, k = 4
Inserisci parola binaria di lunghezza 4: 1010
La parola codificata è: 1011010
```

Test nr. 7 - Decodifica senza errori

```
...
Inserisci parola binaria di lunghezza 7: 1011010
Nessun errore rilevato. La parola decodificata è: 1010
```

Test nr. 8 - Decodifica con singolo errore

```
...
Inserisci parola binaria di lunghezza 7: 1110011
Corretto un errore nella posizione 1. La parola decodificata è: 1011
```

Test nr. 10 - Calcolo distanza di Hamming

```
...
Inserisci prima parola binaria: 10101
Inserisci seconda parola binaria (stessa lunghezza): 10001
La distanza di Hamming tra le due parole è: 1
```

5.2 Testing del programma Prolog

Test nr. 1 - Errore nella scelta del menu

```
-----Benvenuto-----  
...  
Scegliere l'operazione che si desidera eseguire:  
...  
5.  
Opzione non valida. Riprovare.
```

Test nr. 2 - Errore m minore di 2

```
...  
Il bit di parita' deve essere maggiore o uguale a 2.  
Inserire il bit di parita' scelto: 0.  
Input non valido! Deve essere un intero >= 2.
```

Test nr. 4 - Codifica con $m = 3$

```
...  
Inserire il bit di parita' scelto: 3.  
Inserire la parola di lunghezza 4: [1,0,1,0].  
Esecuzione codifica...  
Codice Hamming: [1,0,1,1,0,1,0]
```

Test nr. 7 - Decodifica senza errori

```
...  
Inserire la parola di lunghezza 7: [1,0,1,1,0,1,0].  
Esecuzione decodifica...  
Nessun errore trovato.  
Parola decodificata: [1,0,1,0]
```

Test nr. 8 - Decodifica con singolo errore

```
...  
Inserire la parola di lunghezza 7: [1,1,1,1,0,1,0].  
Esecuzione decodifica...  
Errore trovato in posizione: 2  
Parola decodificata: [1,0,1,0]
```

Test nr. 10 - Calcolo distanza di Hamming

```
...  
Inserire la prima parola: [1,0,1,1,0].  
Inserire la seconda parola: [1,1,0,1,0].  
Distanza di Hamming: 2
```